



# Beyond Linear Regression

While linear models provide a strong baseline, non-parametric methods take a fundamentally different approach — making no assumptions about the underlying functional form. This section explores two families of non-parametric models implemented in `daltoolbox`: **Tree-Based Methods** (`reg_dtree` and Random Forest), which partition the feature space into interpretable regions, and **Neighborhood-Based Methods** (`reg_knn`), which predict based on the proximity of similar observations — all applied to the Boston dataset in R.

TREE-BASED & NEIGHBORHOOD REGRESSION

[R / DALTOOLBOX](#)

[BOSTON DATASET](#)

# Tree-Based Regression

While linear regression provides a strong baseline, tree-based models take a different approach — they partition the feature space into regions, making model decisions interpretable and visual. Building on the foundation established with `reg_lm`, this section covers two tree-based approaches in `daltoolbox`: Decision Trees, which produce explicit rules, and Random Forest, which combines multiple trees for greater predictive robustness.

[TREE-BASED REGRESSION](#)[R / DALTOOLBOX](#)[BOSTON DATASET](#)

# Decision Tree for Regression

A decision tree recursively partitions the feature space using **binary splits** on input features. In the regression setting, each leaf node predicts the **mean of the training samples** that fall into that region. Because it is **non-parametric**, the tree assumes no functional form between inputs and output — making it naturally capable of capturing **non-linearities** while remaining fully interpretable: every prediction can be traced back through a sequence of human-readable rules.

## How It Works

At each node, the algorithm selects the feature and threshold that minimizes the MSE of the resulting split. This continues recursively until a stopping criterion is met. **Prediction** for a new observation = mean of target values in the leaf it falls into.

## Key Hyperparameter

**Tree depth** (`max_depth`) governs the bias-variance trade-off:

- **Shallow tree** → high bias, underfitting
- **Deep tree** → low bias, high variance, overfitting

The gap between train ( $R^2$ : 0.859) and test ( $R^2$ : 0.512) metrics illustrates this directly.

## Strengths & Limitations

### Strengths:

- Fully interpretable decision rules
- Captures non-linearities naturally
- No feature scaling required

### Limitations:

- Prone to overfitting (high variance)
- Unstable — small data changes alter the tree

📖 Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth. — The foundational reference for CART, the algorithm underlying most decision tree implementations.

# reg\_dtree: Decision Tree for Regression

## Setup & Data

```
# Regression Decision Tree

#install.packages("daltoolbox")
library(daltoolbox)
library(MASS)

data(Boston)
Boston <- as.matrix(Boston)

set.seed(1)
sr <- sample_random()
sr <- train_test(sr, Boston)
boston_train <- sr$train
boston_test <- sr$test
```

## Training & Evaluation

```
# Training
model <- reg_dtree("medv")
model <- fit(model, boston_train)

# Evaluation
train_pred <- predict(model, boston_train)
train_eval <- evaluate(model,
  boston_train[, "medv"], train_pred)
print(train_eval$metrics)
# mse=12.68, smape=0.13, R2=0.86

test_pred <- predict(model, boston_test)
test_eval <- evaluate(model,
  boston_test[, "medv"], test_pred)
print(test_eval$metrics)
# mse=29.38, smape=0.16, R2=0.51
```

## Step-by-Step

- Load `daltoolbox` and the `Boston` dataset (MASS)
- Convert to matrix with `as.matrix()` for better performance
- Split into train/test sets using `sample_random()` and `train_test()`
- Define the model with `reg_dtree("medv")` and fit it using `fit()`
- Evaluate with `evaluate()`: metrics include MSE, SMAPE, and  $R^2$
- Compare train ( $R^2=0.86$ ) vs. test ( $R^2=0.51$ ) to detect overfitting — a key signal as we progress to more complex models

📌 **Key Concept:** The decision tree partitions the feature space and estimates the target variable by averaging leaf node values. It is interpretable and captures non-linearities. The main shift from classification lies in the **evaluation step**, not in the fitting logic — a pattern that will carry through the entire regression workflow.

# Neighborhood-Based Regression

Building on the decision tree baseline, we now explore neighborhood-based methods for predictive regression using the `daltoolbox` package in R. KNN (K-Nearest Neighbors) estimates a target variable by computing the **average of the k nearest neighbors** in feature space — a simple, non-parametric approach that effectively captures local patterns without assuming a global model structure.

NEIGHBORHOOD REGRESSION

[R / DALTOOLBOX](#)

[KNN](#)

# K-Nearest Neighbors for Regression

KNN is a **non-parametric, instance-based** learning method. Rather than fitting an explicit model, it stores the entire training set and makes predictions by locating the **k most similar observations** (neighbors) in feature space, then averaging their target values. Because no model is learned during training, KNN is often called a *lazy learner*. Performance is highly sensitive to **feature scaling** and the **choice of k**, both of which directly affect how distances are computed and how the bias-variance trade-off is balanced.

## How It Works

For a new observation, compute the distance (e.g., Euclidean) to every training point. Select the **k nearest** neighbors and predict the **mean of their target values**.

The entire training dataset is stored and queried at prediction time — this is the essence of *lazy learning*. No compression or abstraction of the data occurs during the training phase.

## Key Hyperparameter: k

**k** governs the bias-variance trade-off:

- **Small k** → low bias, high variance (risk of overfitting)
- **Large k** → high bias, low variance (risk of underfitting)

**Feature scaling is critical** — variables with larger magnitudes dominate distance computations, so standardization or normalization must be applied before fitting.

## Strengths & Limitations

### Strengths:

- Conceptually simple with no training phase
- Naturally captures **local, non-linear patterns**
- Adapts to complex decision boundaries

### Limitations:

- Slow at prediction time — scales with dataset size
- Sensitive to irrelevant or redundant features
- No explicit model available for interpretation

① Altman, N. S. (1992). An introduction to kernel and nearest-neighbor nonparametric regression. *The American Statistician*, 46(3), 175–185. — The foundational reference for understanding KNN in the regression context and its statistical properties.

# reg\_knn: K-Nearest Neighbors for Regression

## Setup & Data

```
# Regression KNN

library(daltoolbox)
library(MASS)

data(Boston)
Boston <- as.matrix(Boston)

set.seed(1)
sr <- sample_random()
sr <- train_test(sr, Boston)
boston_train <- sr$train
boston_test <- sr$test

# Training — define k
model <- reg_knn("medv", k=5)
model <- fit(model, boston_train)
```

## Training & Evaluation

```
# Training evaluation
train_pred <- predict(model, boston_train)
train_eval <- evaluate(model,
  boston_train[, "medv"], train_pred)
print(train_eval$metrics)
# mse=27.03, smape=0.16, R2=0.70

# Test evaluation
test_pred <- predict(model, boston_test)
test_eval <- evaluate(model,
  boston_test[, "medv"], test_pred)
print(test_eval$metrics)
# mse=26.95, smape=0.18, R2=0.55
```

## Step-by-Step

- Same data preparation as previous examples (Boston dataset, random split)
- Define the model with `reg_knn("medv", k=5)` — the main parameter is `k`
- The model predicts the mean of the `k` nearest neighbors' target variable values
- Evaluate training and test sets using the same regression metrics
- Notice that training ( $R^2=0.70$ ) and test ( $R^2=0.55$ ) are closer together than with the decision tree — a sign of more stable generalization

📌 **Key Concept:** KNN borrows information from neighboring observations. The parameter `k` controls the bias-variance trade-off: small `k` = high variance; large `k` = high bias. The **Experiment Line remains identical** to that of the decision tree — only the model changes.

# Summary: Non-Parametric Regression Methods

## Unified Experimental Workflow

The same daltoolbox cycle — split → fit → predict → evaluate — applies to both tree-based and neighborhood-based regression. The functions `fit()`, `predict()`, and `evaluate()` work identically for `reg_dtree` and `reg_knn`, making it easy to swap and compare models with minimal code changes.

## Decision Tree vs. KNN

Both methods are non-parametric and capture non-linearities, but differ fundamentally: the **decision tree** partitions the feature space into interpretable regions (train  $R^2=0.86$ , test  $R^2=0.51$ ), while **KNN** predicts from local neighbors with more stable generalization (train  $R^2=0.70$ , test  $R^2=0.55$ ). The decision tree overfits more; KNN generalizes more consistently.

## Key Takeaways

Tree depth (`max_depth`) and number of neighbors (`k`) are the critical hyperparameters controlling the bias-variance trade-off in each method. Feature scaling is essential for KNN but irrelevant for decision trees. Both serve as strong non-parametric baselines before moving to ensemble methods like Random Forest.

# References & Additional Sources

Academic references and resources to explore Decision Tree and KNN regression further.

## Online Resources

### CRAN Package – daltoolbox

[cran.r-project.org/web/packages/daltoolbox](https://cran.r-project.org/web/packages/daltoolbox)

### DAL Toolbox – GitHub

[github.com/cefet-rj-dal/daltoolbox](https://github.com/cefet-rj-dal/daltoolbox)

### Boston Dataset (MASS package)

[cran.r-project.org/web/packages/MASS](https://cran.r-project.org/web/packages/MASS)

Boston Housing dataset used in this analysis.

## Key Academic References

- Breiman, L., Friedman, J., Olshen, R. & Stone, C. (1984). Classification and Regression Trees. *Wadsworth*. — Foundational reference for CART decision trees.
- Altman, N. S. (1992). An introduction to kernel and nearest-neighbor nonparametric regression. *The American Statistician*, 46(3), 175–185. — Key reference for KNN regression.
- Harrison, D. & Rubinfeld, D. L. (1978). Hedonic prices and the demand for clean air. *Journal of Environmental Economics and Management*, 5, 81–102. — Original paper introducing the Boston Housing dataset.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning. 2nd ed. *Springer*. — Comprehensive coverage of both decision trees and KNN in a unified statistical framework.
- Ogasawara, E., Sa, A., Castro, A., et al. (2026). daltoolbox: Leveraging Experiment Lines to Data Analytics. R package version 1.3.767. *CRAN*.